

FitBuddy – AI Fitness Plan Generator

Performance Testing Analysis & Test Plan

Based on the uploaded FitBuddy project documentation and the supplied Performance Testing submission template.

1. Testing Overview

Field	FitBuddy Analysis
Testing Tool Used	JMeter / Locust / k6 / Postman – tool is not specified in the FitBuddy source. Select the tool actually used for execution.
Type of Testing	Load Testing and Stress Testing are appropriate for the documented web/API architecture. Actual executed tests to be defined.
Target Module / API	/generate-workout, /submit-feedback, /view-all-users, and / (home page).
Test Environment	FitBuddy is documented for local FastAPI + Uvicorn execution at http://127.0.0.1:8000. Cloud/hosted environment to be defined.
Test Date	Not available in the FitBuddy source.

2. FitBuddy Performance Test Scope

FitBuddy is a FastAPI web application integrated with Google Gemini models. The main performance-sensitive flow is workout generation because the route invokes Gemini 1.5 Pro and Gemini Flash and then stores the result in SQLite. Feedback submission also invokes Gemini 1.5 Pro, while the admin page reads stored user/plan data. The project documentation specifically identifies real-time web interaction as a model requirement.

Scenario	Endpoint / Feature	Performance Focus
1	/	Homepage rendering and form response.
2	/generate-workout	AI generation latency, request throughput, error rate, DB write overhead.
3	/submit-feedback	AI revision latency, DB read/update overhead, error handling.
4	/view-all-users	Database read and HTML rendering as user/plan records increase.

3. Test Scenarios

S.No	Test Scenario / Description	Virtual Users	Duration (sec)	Expected Outcome
1	Baseline load on /generate-workout with valid user details.	5	60	Requests complete successfully; response time < 2 seconds
2	Moderate load on /generate-workout with concurrent users.	10	120	Stable service with low error rate and no sustained high CPU or memory usage
3	Feedback update load on /submit-feedback.	10	120	Plans are updated successfully; Gemini and data is persisted
4	Stress test combining generation, feedback, and admin requests.	20	120	System should degrade gracefully without excessive errors

Note: The virtual-user counts and durations above are proposed test scenarios for the submission document, not recorded execution results.

4. Performance Test Metrics

S.No	Metric	Target Value	Actual Value	Status
1	Response Time (Avg)	< 2 seconds	Not executed / not provided	Pending
2	Response Time (Max)	< 5 seconds	Not executed / not provided	Pending
3	Throughput (Req/sec)	Measure during load test	Not executed / not provided	Pending
4	Error Rate	< 1%	Not executed / not provided	Pending
5	CPU Utilization	< 80%	Not executed / not provided	Pending
6	Memory Utilization	< 80%	Not executed / not provided	Pending

The target thresholds above follow the supplied Performance Testing template. They are acceptance targets, not measured FitBuddy results. The source documentation does not contain benchmark measurements.

5. Suggested JMeter / Locust Execution Procedure

1. Start FitBuddy locally using the documented FastAPI/Uvicorn command.
2. Confirm the application is reachable at <http://127.0.0.1:8000> and the FastAPI docs at </docs>.
3. Create a load-test request for POST </generate-workout> with valid form fields: username, user_id, age, weight, goal, and intensity.
4. Create a second request for POST </submit-feedback> using a valid user_id and feedback.
5. Add a GET request for </view-all-users> to measure database retrieval and page rendering.
6. Run a baseline test first, then moderate load, then stress load.
7. Capture average/max response time, throughput, error rate, CPU utilization, and memory utilization.
8. Attach the JMeter/Locust/k6 result screenshots to the final submission.

6. FitBuddy Performance Analysis

Area	Analysis
AI latency	/generate-workout invokes Gemini 1.5 Pro for the structured 7-day plan and Gemini Flash for nutrition tips. External AI API la
Feedback latency	/submit-feedback retrieves the original plan, sends original plan + feedback to Gemini 1.5 Pro, updates SQLite, and renders
Database	SQLite with SQLAlchemy is used for persistence. Under higher concurrent write loads, database contention should be obser
Frontend rendering	Jinja2 templates render index.html, result.html, and all_users.html. Response time should be checked separately from AI ge
Admin view	/view-all-users reads user and plan records. Performance should be tested with increasing numbers of stored records.
Resource usage	CPU and memory should be monitored during concurrent AI requests and database operations. The FitBuddy source does n

7. Observations & Analysis

Based on the architecture documented for FitBuddy, the principal performance dependency is the AI generation layer rather than the HTML/Jinja2 rendering alone. Workout generation and feedback updates depend on Gemini API calls, while user and plan persistence uses SQLite/SQLAlchemy. Therefore, performance testing should distinguish application processing time from external model/API latency where the selected testing tool permits it. The supplied project documentation confirms that local deployment and verification are part of the project workflow, but it does not report numerical benchmark results.

Result interpretation should be completed only after an actual load/stress test run. If measured average response time is below 2 seconds, maximum response time below 5 seconds, error rate below 1%, and CPU/memory remain below 80%, the corresponding template criteria can be marked Pass. Otherwise, record Fail and include the measured value and cause/workaround.

8. Screenshots / Evidence

Attach the following evidence after executing the tests: (1) load-test summary, (2) response-time graph, (3) throughput graph, (4) error-rate report, (5) CPU utilization, (6) memory utilization, and (7) terminal/browser evidence showing FitBuddy running locally. The supplied performance template explicitly asks for screenshots of the testing-tool output.

Evidence No.	Screenshot / Evidence to Attach	Status
1	JMeter / Locust / k6 test summary	To be attached after execution
2	Average and maximum response-time graph	To be attached
3	Throughput / requests-per-second graph	To be attached
4	Error-rate report	To be attached
5	CPU and memory monitoring screenshot	To be attached

9. Conclusion

FitBuddy has a clear performance-testing scope because its core user journeys combine FastAPI routing, Gemini AI generation, SQLite persistence, and Jinja2 rendering. The most important endpoints for performance evaluation are /generate-workout and /submit-feedback, with /view-all-users providing a database/rendering workload. This document maps those components to the supplied five-mark Performance Testing format. Actual performance results must be inserted from a real test run; no benchmark values have been fabricated.

Source & Scope Note

This PDF uses the uploaded Performance Testing template for structure and the uploaded FitBuddy documentation for application-specific analysis. The template defines targets such as average response time <2 seconds, maximum response time <5 seconds, error rate <1%, CPU <80%, and memory <80%. The FitBuddy source describes FastAPI, Gemini Pro/Flash, SQLite/SQLAlchemy, Jinja2, routes, and local deployment, but does not provide actual performance measurements.